AI-Driven Integration Testing for Deterministic State Machine Breakpoints

Ping Gong
asd28269@gmail.com
Independent Researcher
China

ABSTRACT

Large language models (LLMs) have dramatically improved code generation efficiency, yet system-level verification still depends heavily on manually written test cases. As development accelerates and inter-module dependencies grow increasingly complex, traditional approaches face structural bottlenecks. This paper presents DSMB-Agent, an agent-driven integration testing method for complex systems that shifts the majority of system-level verification work from developers to an autonomous agent through a three-phase process: *Understand–Design–Execute*. The method is built on two core mechanisms: (1) a *tree-structured command space* that exposes the system under test as a hierarchical `module`→`command`→`parameter` tree, where the agent can both use existing commands and register new ones at runtime to extend its testing capabilities; and (2) *step-wise reasoning* with a breakpoint controller that splits execution into single steps, allowing the agent to observe causal chains after each step and dynamically adjust its testing direction. On a prototype event-driven game server with 6 business modules and 7 seeded defects, we design three autonomy levels (L0/L1/L2) and a 2×2 ablation study, conducting 3 repeated runs on each of two LLMs (GLM-5.1 and GPT-5.4) for a total of 42 experiment runs. Results show that on GLM-5.1, the step-wise execution group discovers an average of 3.7/7 defects versus 1.3/7 for batch execution; on GPT-5.4, the code+batch group performs best (5.0/7); and in L0 mode both models can register a complete test command set from scratch and discover 3–5 defects, while L1 mode on GPT-5.4 achieves a single-run maximum of 6/7, validating the method's cross-model feasibility.

KEYWORDS

Integration Testing, LLM Agent, Command Tree, Step-Wise Reasoning, Autonomy Levels, Ablation Study

1 INTRODUCTION

1.1 Problem and Motivation

Large language models have produced a step-change in efficiency for code generation, interface implementation, refactoring, and local bug fixing [3, 12]. The direct consequence is that system functionality grows faster and module boundaries shift more frequently. Yet the verification side has not kept pace—many teams still rely on manual code review and hand-written integration test cases as their safety net. When AI accelerates development by several fold, manual review and hand-enumerated test steps become increasingly unable to match the new pace of production. More fundamentally, the bottleneck of manual review lies not in review speed but in the upper bound of human cognitive load [6].

The traditional integration testing paradigm works as follows: developers first understand the business logic, then translate verification logic into a set of explicit test cases—specifying system entry points, writing operation sequences, choosing input parameters, enumerating boundary conditions, and defining expected outcomes. In systems with few modules and clear dependencies, this paradigm remains effective. However, in complex systems, the real risks often lie not in individual interface return values but in inter-module interactions: a single local state change may propagate along event, message, and callback chains to multiple subsystems. These interactions are simultaneously *long-path* (three or more layers of event propagation), *large-state-space*, and *latent* (manifesting only at runtime). Even developers with deep domain knowledge find it difficult to exhaustively enumerate all cross-module impact paths when writing test cases.

When the inter-module interaction complexity exceeds the upper bound of manual enumeration, the failure of traditional methods is not accidental but structural.

1.2 Core Idea

This paper proposes shifting system-level verification from "manually writing test steps" to "agent-driven autonomous exploration." Developers provide source code, requirement documents, and safety boundaries; the agent is responsible for understanding system structure, identifying module interactions, choosing exploration paths, generating verification commands, executing tests, and producing reports. The human role shifts from "test step writer" to "rule provider and result reviewer."

To enable this shift, we propose two core designs:

- **Tree-Structured Command Space**—inspired by the hierarchical design of CLIs, the system under test is exposed as a `module`→`command`→`parameter` command tree. The agent interacts with the tree via WebSocket + JSON and can not only use existing commands but also register new ones at runtime through `register_cmd`, dynamically extending its testing capabilities.
- **Step-Wise Reasoning**—a breakpoint controller separates the enqueuing and execution of operations.

Source code and experiment data: https://github.com/gongp0615/ai-driven-integration-testing.

After each step executes, the world is *held* (Hold World). During this pause, the agent has full autonomous decision-making authority: it can observe and reason, query state, modify data, register new interfaces, or read source code, actively constructing preconditions for the next test step before deciding when to advance the world.

1.3 Contributions

The main contributions of this paper are:

(1) We propose the tree-structured command space and runtime command registration mechanism, enabling the agent to discover, use, and extend the system's test interfaces (Sections 3.1–3.2).
(2) We propose step-wise reasoning with the Hold World mechanism, pausing the system world after each step so the agent can autonomously decide—query, modify, register interfaces, or read code—before advancing (Section 3.3).
(3) We design three autonomy levels (L0/L1/L2) and a 2×2 ablation study, conducting 3 repeated runs on two LLMs to quantify the individual contributions of command registration and step-wise execution (Sections 4–5).
(4) In 42 runs on an event-driven system prototype, GLM-5.1's step-wise group discovers an average of 3.7/7 defects (vs. batch group 1.3/7), GPT-5.4's code+batch group performs best (5.0/7), and L1+LSP on GPT-5.4 reaches a single-run maximum of 6/7 (Section 5).

1.4 Paper Organization

Section 2 reviews related work; Section 3 details the method design; Section 4 describes the experimental setup; Section 5 reports results; Section 6 discusses findings and implications; Section 7 analyzes threats to validity; Section 8 concludes and outlines future work.

2 RELATED WORK

2.1 Model-Based Testing

Model-based testing (MBT) generates test cases automatically by constructing abstract models (state machines, UML models, etc.) of the system under test [8, 14]. The core advantage of MBT is systematic path coverage derived from models, but its bottleneck is model construction itself: developers must manually maintain consistency between the model and the implementation, and models typically cannot capture runtime dynamic behavior. Our command tree can be viewed as a lightweight operational model, but the key distinction is that the command tree is directly exposed from the running system, requires no separate modeling effort, and the agent can dynamically extend the model at runtime.

2.2 Automated Exploratory Testing

Tools such as Monkey [7], Sapienz [11], and Stoat [13] in mobile app testing automatically explore GUI state spaces using random or model-based strategies. These methods align directionally with our agent-driven exploration—both seek to shift testing from predefined scripts to dynamic exploration. However, GUI exploratory testing primarily targets UI-level state coverage, whereas our work focuses on server-side inter-module event propagation and state interactions, with different testing objectives and observability mechanisms.

2.3 LLMs for Software Testing

The application of LLMs in software testing has grown rapidly in recent years. ChatUniTest [4] uses LLMs to generate unit tests; CodaMosa [9] combines LLMs with search algorithms to improve code coverage; TitanFuzz [5] uses LLMs for fuzz testing of deep learning libraries. These works primarily focus on automated generation of unit tests or API-level tests, while our work targets cross-module behavior verification at the integration testing level. Furthermore, in the above works the LLM's role is "one-shot test code generation," whereas in our approach the agent continuously interacts with the system under test at runtime, dynamically adjusting its testing strategy based on observations.

2.4 Fuzzing

Coverage-guided fuzz testing (e.g., AFL [15], libFuzzer [10]) uses code coverage feedback to guide input generation. Our step-wise reasoning shares a conceptual connection with coverage-guided fuzzing in "adjusting exploration direction based on feedback," but with two fundamental differences: (1) fuzzing's feedback signal is code coverage, whereas ours is event propagation chains and business logs—the latter carries semantic information enabling causal reasoning rather than mere path coverage; (2) fuzzing generates low-level byte sequences, whereas our agent operates structured commands with business semantics.

2.5 Adaptive and Online Testing

Adaptive testing strategies [1, 2] dynamically select subsequent test cases based on results of previously executed tests. Our step-wise reasoning mechanism aligns with this direction, but adaptive testing typically selects from a predefined test pool, whereas our agent can generate entirely new test actions at runtime (including registering new commands), with an exploration space unconstrained by any predefined pool.

3 METHOD DESIGN

We frame agent-driven integration testing as three iterable phases: **Understand** → **Design** → **Execute**. The two core mechanisms—tree-structured command space and step-wise reasoning—are woven throughout these phases.

3.1 Tree-Structured Command Space

3.1.1 Structure Definition. The system under test is exposed as a structured command tree. The agent establishes a persistent WebSocket connection with the system, and all interactions use JSON format. Each command follows a uniform three-level hierarchy:

```
cmd (module) -> action (command) -> params (parameter
     table)
```

Using our experimental prototype as an example, the command tree structure is:

```
bag
  AddItem      {itemId: int, count: int}
  RemoveItem   {itemId: int, count: int}
  query        {itemId?: int}
task
  query        {taskId?: int}
equipment
  Equip        {slot: string, itemId: int}
  Unequip      {slot: string}
  query        {}
signin
  CheckIn      {day: int}
  ClaimReward  {day: int}
  query        {day?: int}
mail
  ClaimAttachment {mailId: int}
  query        {mailId?: int}
system
  next         {}        // step-wise execution
  batch        {}        // batch execution
  help         {}        // query command tree
  register_cmd {name, target, action}
  listcmd      {}
player
  login        {playerId: int}
```

The corresponding JSON request format is:

```
{"cmd":"bag","action":"AddItem",
 "params":{"itemId":2001,"count":1}}
```

The agent can query the complete command tree via `{"cmd":"system","action":"help"}`, achieving runtime *discoverability*.

3.1.2 Extension to Microservice Architectures. This structure naturally extends to a four-level hierarchy: **service → module → command → parameters**. After obtaining the service list from a registry, the agent explores layer by layer:

```
{"service":"game-server","cmd":"bag",
 "action":"AddItem","params":{...}}
```

3.2 Runtime Command Registration

If all test commands are predefined by humans, the agent cannot bridge the gap when "existing interfaces are insufficient to verify a suspicion." The `register_cmd` mechanism allows the agent to register new test commands within controlled boundaries:

Registration flow:

(1) The agent issues a registration request:

```
{"cmd":"system","action":"register_cmd",
 "params":{"name":"test_remove_negative",
  "target":"bag","action":"RemoveItem"}}
```

(2) The system performs safety checks—verifying the name does not conflict with built-in commands and the target/action combination is on the allowed whitelist (`isAllowedRawBinding`).

(3) After successful registration, the agent can invoke the underlying business method directly through the new command:

```
{"cmd":"test_remove_negative","action":"exec",
 "params":{"itemId":2001,"count":-1}}
```

Safety constraints: Each WebSocket connection maintains independent session state, with custom command registration isolated at the session level; the system enforces localhost binding (`127.0.0.1`), allowing only local process access.

The key value of this mechanism is that the built-in command layer typically includes parameter validation (e.g., `RemoveItem` rejects negative `count`), but the underlying business method may lack equivalent protection. By registering raw interface commands, the agent can verify defects masked by the command layer.

3.3 Step-Wise Reasoning and Hold World

3.3.1 Motivation. Batch execution mode submits all operations at once, leaving the agent to observe only the final cumulative state—essentially facing the same dilemma as manually pre-written test cases: all operations must be determined before execution, with no ability to adjust direction based on intermediate observations.

The focus of step-wise reasoning is not on "finer granularity" but on creating autonomous thinking and decision-making space for the agent through **Hold World**.

3.3.2 Mechanism: Breakpoint Controller and Hold World. The **breakpoint controller** implements separation of operation enqueuing and execution:

(1) When the agent sends a business command, the operation is enqueued to the pending queue, and **the system world undergoes no changes**.

(2) When the agent sends `{"cmd":"system","action":"next"}`, the breakpoint controller dequeues and executes one operation; all resulting event propagation and logs are captured by the event bus.

(3) `next` returns the **complete log chain** for this execution, after which **the world pauses again**, awaiting the agent's next decision.

The key design here is **Hold World**: between when `next` returns its result and when the agent issues its next instruction, the system under test is in a completely quiescent state—no timers advancing, no asynchronous events firing, no background state changes. This pause is not mere waiting but a window in which the agent has full autonomous decision-making authority. During this period, the agent can perform any combination of:

- **Observe and query**—check the current state of any module via `query` commands, confirming whether the side effects of the previous step match expectations.
- **Modify system state**—inject data or modify state via commands (e.g., adding/removing items, triggering check-ins), artificially constructing specific preconditions to observe subsequent behavior.
- **Register new commands**—dynamically create new test interfaces via `register_cmd`, bypassing command-layer validation to directly invoke underlying methods.
- **Read source code**—examine code implementation via `read_file` and `search_code`, verifying whether runtime-observed behavior matches the code logic.
- **Update knowledge**—record current findings via `update_knowledge`, avoiding redundant exploration of known paths.

Only when the agent proactively issues `next` does the world advance one step. This means the agent has complete control over "when to advance the world"—it can issue multiple query and modification commands after a single `next` to construct complex test scenarios, then advance to the next step to observe system reactions.

After each step, the agent undergoes a complete reasoning cycle:

```
next (world advances one step)
  -> Observe: which events fired? Is the log
              chain complete?
  -> Judge:   does behavior match expectations?
              any anomalies?
  -> [Hold World: agent decision window]
      * Query -- check related module state
      * Modify -- inject data for preconditions
      * Register -- create new commands
      * Read -- examine source code
  -> Enqueue next command
  -> next (world advances again)
```

In contrast, {`"cmd":"system","action":"batch"`} executes all pending operations at once. In batch mode the world runs continuously, and the agent loses not only observation granularity but also the ability to actively intervene in system state, construct test conditions, and adjust exploration direction between steps.

3.3.3 Example: How Hold World Guides Exploration. **Step 1** — Agent enqueues `bag.AddItem({itemId:2001, count:1})`, executes `next`. Observes log: `[Bag] add` → `[Task] progress+1` → `[Task] completed` → `[Achievement] unlocked`. [**Hold World**] Agent thinks: "Task 3001 is completed. If I add the same item again, will the completed task be re-triggered?"

Step 2 — Agent decides to repeat the test, enqueues the same command, executes `next`. Observes: `[Task] progress+1 (now 2/1)` → `[Task] completed`. [**Hold World**] Agent thinks: "The completed task was re-triggered." → Confirms defect B2. Agent uses the pause period to execute `achievement.query` to check achievement module state → discovers achievements were also re-unlocked → confirms the cascading impact of B2.

Step 3 — Agent tries `bag.RemoveItem({count:-1})`, rejected by the command layer. [**Hold World**] Agent decides to register a new command: executes `register_cmd` to create `test_remove_negative`. After successful registration, enqueues `test_remove_negative.exec({itemId:2001, count:-1})`, executes `next`. Observes: item count changes from 1 to 2 → Confirms defect B1.

In this process, the agent made different types of decisions at each pause window: repeating operations to verify idempotency, querying related modules to trace cascading effects, and registering new commands to break through interface limitations. These decisions were not pre-planned but dynamically generated during Hold World windows based on observations.

3.3.4 Implementation Considerations for Different Server Architectures. Step-wise execution requires inserting pause points at the boundary of "a single business operation's serialized execution." The key constraint is that **step does not insert write operations within a serialized execution flow**—it pauses the world only after a complete synchronous execution chain concludes. When the execution encounters suspension points such as RPC calls or asynchronous messages, the current execution flow has already yielded control; these suspension points are themselves natural serialization boundaries where step can intervene.

In practice, the step-wise mechanism must adapt to the underlying actor model. Whether the actors are explicit (e.g., Skynet) or implicit (e.g., Go goroutine + channel), server-side concurrency units typically follow the "exclusive state + message-driven" pattern, which naturally aligns with step-wise pause semantics:

Skynet (explicit actor): The concurrency unit that the AI needs to intervene in is each *service*. Each service owns independent state and a message queue; regardless of which worker_thread drives it, execution continues within the same logical workspace. The breakpoint controller inserts pause points at service-level message processing boundaries—after one message is fully processed (including all synchronous callbacks it triggers), the world pauses until the agent decides to deliver the next message. When message processing initiates an RPC (via `skynet.call`, which suspends the current coroutine), that suspension point is also a valid pause boundary—the service's state is consistent at that moment, and step can safely intervene.

Go server (implicit actor): In Go, there is no fixed mapping between goroutines and logical entities—the same business flow may span multiple goroutines, and a single goroutine may serve different requests in sequence. However, when system design centers on "state ownership + message flow" (each logical entity receives instructions through a dedicated channel), this is essentially an implicit actor model. The breakpoint controller governs the timing of channel message delivery rather than goroutine scheduling. Our experimental prototype adopts this approach—operation enqueuing and execution are separated via a WebSocket message queue and

breakpoint controller, making goroutine scheduling transparent to the step-wise mechanism.

General principle: Regardless of the specific form of the underlying actor model, the implementation concerns for step-wise execution are the same: (1) identify the serialization boundaries of actor message processing (including synchronous execution completion and RPC suspension yield points); (2) insert pause points at those boundaries; (3) ensure the actor's state is completely quiescent during the pause, accepting no new messages.

3.4 Three-Phase Process

Understand Phase: The agent's goal is to build a structural understanding of system module relationships and event flows. In real engineering projects, source code scale typically far exceeds the agent's context window limit, making file-by-file reading infeasible. Therefore, the understanding phase should prioritize *semantic-level tools* over text-level search:

- **LSP (Language Server Protocol)**—the agent obtains precise semantic information through a locally deployed language server: `go to definition` to locate symbol implementations, `find references` to trace event publish/subscribe relationships, `workspace/symbol` for module-level symbol indexing, and `call hierarchy` for call chain analysis. LSP provides compiler-analyzed structured results with far higher precision than text search.
- **Static AST analysis**—optionally, code analyzers can pre-generate module structure summaries (event pub/sub tables, function signatures, cross-module dependency graphs) as the agent's initial knowledge.
- **Text search**—`read_file` and `search_code` serve as supplementary tools for scenarios where LSP is less effective (e.g., comments, configuration files, string constants).

Our experimental prototype, with its small module scale (6 files), directly used text-level tools. In real projects, however, LSP is the core tool for the understanding phase—it enables the agent to precisely locate key module interactions and event propagation paths without reading all source code.

Design Phase: When existing commands are insufficient to verify a suspicion, the agent registers new test commands within the allowed rules. This phase bridges the gap between "seeing something suspicious in code but being unable to construct triggering input."

Execute Phase: The agent explores step by step via step-wise execution, continuously refining its understanding and actions during execution: enqueue operation → step-wise execute → causal tracing → path expansion → knowledge update. The three phases iterate repeatedly.

4 EXPERIMENTAL SETUP

4.1 System Under Test

The experimental prototype is an event-driven game server implemented in Go, containing 6 business modules and 2 infrastructure modules. We chose this system because event-driven systems naturally exhibit characteristics typical of complex systems: dense inter-module relationships, frequent state changes, and chain side effects triggered by business actions.

Table 1: Module overview of the experimental prototype.

Module	Responsibility	Key Events
Bag	Item add/remove	Publishes `item.added/removed`
Task	Task progress	Subscribes `item.added`; publishes `ta`
Achievement	Unlock achievements	Subscribes `task.completed`, `item.ac`
Equipment	Equip/unequip	Subscribes `item.added`; publishes `eq`
SignIn	Daily check-in	Publishes `signin.claimed`
Mail	Mail & attachments	Subscribes `achievement.unlocked`,
Event Bus	Event routing	Global event routing & log capture
Breakpoint Ctrl	Step control	Enqueue, step-execute, log collectio

Table 2: Seeded defects in the prototype.

ID	Module	Level	Severity	Description
B1	Bag	L1	Critical	`RemoveItem` lacks `count`≤0 check; neg
B2	Task	L2	High	Completed task can re-fire completio
B3	Achievement	L2	Medium	`collector_100` counts wrong objects
B4	SignIn	L3	High	`ClaimReward` has no idempotency gua
B5	Equipment	L3	Critical	Equipped item not removed from bag
B6	Mail	L4	High	`mail.claimed` event has no subscribe
B7	SignIn+Equip	L4	Medium	Day-7 reward ID collides with weapo

4.2 Seeded Defects

Seven defects are seeded in the prototype across 4 difficulty levels. Defect design draws on common fault patterns in event-driven and microservice systems [16], including missing state guards, lack of idempotency, event chain breaks, and cross-module ID conflicts.

4.3 Autonomy Levels

We design three autonomy levels to evaluate the effects of different capability combinations:

L0 (Fully Autonomous): The agent receives no pre-built business commands; it has only infrastructure commands (`player.login`, `system.next`, `system.batch`, `system.help`) and command registration capability. The agent must read source code and register all test commands itself.

L1 (Semi-Autonomous): The agent has all pre-built business commands and retains command registration capability. When built-in commands cannot cover a test path, the agent can register new commands to supplement.

L2 (Assisted Execution): All business commands are pre-built by humans; the agent has no command registration capability. Within L2, a 2×2 ablation study distinguishes the individual contributions of code understanding and step-wise execution:

Table 3: L2 ablation study design.

Group	Code Access	Step-Wise	Description
A (batch-only)	No	No	Lowest autonomy baseline
B (step-only)	No	Yes	Step-wise execution contribution
C (code-batch)	Yes	No	Code understanding contribution
D (dual)	Yes	Yes	Both factors combined

Table 4: Experimental configuration.

Parameter	GLM-5.1	GPT-5.4
Provider	Zhipu AI (Anthropic-compat. API)	OpenAI (Codex CLI)
Max iterations	110	80
L2 runs per group	3	3
L0 runs	3	3
L1 runs	3	3
Connection	WebSocket	WebSocket
Code tools	`read_file` + `search_code`	+ LSP (gopls)

Table 5: Evaluation metrics.

Metric	Definition
Bug Recall	True defects found / 7
Bug Precision	True defects / total reported defects

Additionally, **code-only** serves as a pure static analysis baseline, where the agent produces reports solely by reading source code without runtime interaction.

4.4 Experimental Configuration

Experiments are conducted on two LLMs to assess the method's dependence on the underlying model:

4.5 Evaluation Metrics

Table 6: L2 defect detection matrix (GLM-5.1, 3 runs). "·" = 0/3.

Defect	Level	A	B	C	D	code-only
B1	L1	·	·	2/3	·	1/3
B2	L2	1/3	·	1/3	1/3	·
B3	L2	·	2/3	1/3	·	3/3
B4	L3	3/3	3/3	3/3	3/3	3/3
B5	L3	·	3/3	1/3	1/3	2/3
B6	L4	·	3/3	3/3	3/3	3/3
B7	L4	·	·	·	·	·

Table 7: L2 summary metrics (GLM-5.1, mean of 3 runs).

Metric	A	B	C	D	code-only
Avg defects found	1.3	3.7	3.7	2.7	4.0
Avg iterations	55	87	72	93	25
B1 detection rate	0%	0%	67%	0%	33%

5 EXPERIMENTAL RESULTS

5.1 L2 Ablation Study (GLM-5.1, 3 Runs)

Table 6 reports the defect detection matrix for the four L2 groups and the code-only baseline on GLM-5.1.

5.2 L2 Ablation Study (GPT-5.4, 3 Runs)

To verify whether findings from GLM-5.1 hold across models, we repeated the L2 ablation on GPT-5.4 with 3 runs per group. GPT-5.4 experiments also integrated LSP (gopls) for semantic code analysis.

Table 8: L2 defect detection matrix (GPT-5.4, 3 runs).

Defect	Level	A	B	C	D	code-only
B1	L1	·	·	·	·	2/3
B2	L2	·	2/3	3/3	3/3	2/3
B3	L2	3/3	3/3	3/3	3/3	2/3
B4	L3	·	·	3/3	3/3	2/3
B5	L3	1/3	1/3	2/3	·	2/3
B6	L4	·	·	3/3	3/3	2/3
B7	L4	1/3	·	1/3	·	·

Table 9: L2 summary metrics (GPT-5.4, mean of 3 runs).

Metric	A	B	C	D	code-only
Avg defects found	1.7	2.0	5.0	4.0	4.0
Avg iterations	6	54	31	61	—
B1 detection rate	0%	0%	0%	0%	67%

5.2.1 Key Findings. **Finding 1: Step-wise execution is most effective on GLM-5.1; code understanding is most effective on GPT-5.4.** On GLM-5.1, groups B (step-only, 3.7/7) and C (code-batch, 3.7/7) tie as the best, both substantially ahead of group A (batch-only, 1.3/7). On GPT-5.4, group C (code-batch, 5.0/7) performs strongest. This indicates that step-wise execution and code understanding each make independent contributions, but the marginal return of code understanding grows with model capability.

Finding 2: B1 is difficult to detect reliably in runtime groups. B1 (`RemoveItem` with negative arguments) was not detected in any GPT-5.4 L2 runtime group and was only sporadically detected in GLM-5.1's code-batch group (2/3). Only L0/L1 modes with command registration can reliably reproduce B1, validating the necessity of the command registration mechanism for overcoming interface blind spots.

Finding 3: Both models exhibit negative interaction effects. On GLM-5.1, group D (dual, 2.7/7) underperforms the mean of groups B and C (3.7/7); on GPT-5.4, group D (4.0/7) similarly underperforms group C (5.0/7). The cause: simultaneously enabling code analysis and step-wise execution forces the agent to switch between modes, increasing cognitive load and iteration consumption, performing worse than focusing on a single factor.

Finding 4: code-only has the highest cross-model stability. code-only achieves approximately 4/7 defect detection rate on both models, with the highest B1 detection probability among L2 groups (1/3 on GLM-5.1, alongside code-batch's 2/3). However, its inherent limitation is the inability to confirm or rule out false positives through runtime verification.

Finding 5: The most easily detected defects differ between models. On GLM-5.1, B4 (sign-in reward without idempotency guard) was detected in all 15 L2 runs; B6 (mail attachment chain break) in 12/15. On GPT-5.4, B3 (achievement counting wrong objects) was the most stable (14/15), while B4 and B6 were primarily found in groups with code access (8/15 each). The common trait is that these defects exhibit behavioral anomalies highly visible in logs or code.

5.3 Cross-Model Comparison

Table 10 merges results from both models (all as 3-run means).

Table 10: Cross-model L2 ablation comparison (mean of 3 runs).

Group	GLM-5.1	GPT-5.4	Trend
A (batch-only)	1.3	1.7	Consistent: weakest
B (step-only)	3.7	2.0	GLM better
C (code-batch)	3.7	5.0	GPT better
D (dual)	2.7	4.0	GPT better
code-only	4.0	4.0	Consistent

Core insight: Step-wise execution and code understanding each make independent contributions, but their relative magnitudes depend on model capability. On GLM-5.1, the marginal benefit of step-wise execution is more pronounced (B group 3.7 vs A group 1.3), while on GPT-5.4, code understanding dramatically improves batch mode performance (C group 5.0 vs A group 1.7). Both models exhibit negative interaction effects (D group underperforms single-factor groups), indicating that simultaneously enabling both capabilities creates cognitive load issues under the current agent architecture. This implies that as LLM capabilities improve, the optimal strategy should adjust the weight allocation between code understanding and runtime exploration based on model characteristics.

5.4 L0 Experiment Results

In L0 mode, the agent starts from scratch with no pre-built business commands.

Table 11: L0 experiment results.

Metric	GLM-5.1			GPT-5.4		
	run1	run2	run3	run1	run2	run3
Iterations	102	100	108	80	70	71
Registered cmds	7	7	7	7	7	7
Defects	4/7	4/7	3/7	5/7	4/7	4/7

GLM-5.1 averages 3.7/7, GPT-5.4 averages 4.3/7. In all 6 L0 runs across both models, the agent successfully registered a complete test command set (7 commands) and discovered B1—a defect that L2 groups on both models were nearly unable to reach. B4 and B6 were reliably detected in all runs. These L0 results validate the cross-model feasibility of fully autonomous mode.

5.5 L1 Experiment Results

The core validation goal of L1 mode is the "interface gap" hypothesis. In all 6 runs, the agent autonomously registered raw interface commands to bypass command-layer validation. In GLM-5.1 run2, after registering `test_remove_negative`, the agent observed item count change from 3 to 104; in GPT-5.4 run2, the same command registration revealed item count changing from 1 to 2—**both models successfully reproduced defect B1 through command registration**.

Table 12: L1 experiment results.

Metric	GLM-5.1			GPT-5.4		
	run1	run2	run3	run1	run2	run3
Iterations	105	108	107	66	87	75
Registered cmds	2	2	2	2	1	1
Defects	4/7	4/7	2/7	5/7	6/7	5/7

GLM-5.1 averages 3.3/7, GPT-5.4 averages 5.3/7. GPT-5.4's L1 performance significantly exceeds GLM-5.1's, because GPT-5.4 combined with LSP tools can more efficiently understand code structure, achieving broader module coverage within limited iterations. Notably, GPT-5.4 run2 discovered 6/7 defects (including B7), the highest single-run record across all experiments. B4 and B6 were reliably detected in all 6 runs across both models.

6 DISCUSSION

6.1 Command Registration Is Key to Overcoming Interface Blind Spots

The B1 detection rate in L2 runtime groups is extremely low (GLM-5.1: only code-batch at 2/3; GPT-5.4: all at 0/3), while L0/L1 can reliably reproduce this defect through command registration. This comparison shows that when test interfaces cannot construct the inputs needed to trigger a defect, no amount of exploration strategy can compensate for the interface's inadequacy. The command registration mechanism upgrades the agent from "command user" to "command designer," and is the core differentiator of our method from traditional automated testing.

6.2 Hold World Empowers Active Agent Intervention

The advantage of step-wise execution over batch execution lies not merely in finer granularity or causal traceability, but fundamentally in the fact that **Hold World** creates autonomous decision-making space for the agent. After each step, the world is completely quiescent, and the agent can freely query state, modify data, and register new interfaces—actively constructing test conditions rather than passively awaiting results. In batch mode, the agent must determine all operations before execution; the world runs continuously, and the agent loses not only observation granularity but also the ability to actively intervene in system state, construct preconditions, and adjust exploration direction between steps. GLM-5.1 experimental data directly illustrates this: group B (step-wise, 3.7/7) substantially outperforms group A (batch, 1.3/7), and group B stably detected B4, B5, and B6 across all 3 runs—the agent decided during pauses to query related modules to trace cascading effects, repeat operations to verify

idempotency, and actively construct boundary conditions. These decisions were dynamically generated in Hold World windows, not pre-planned.

6.3 Code Understanding Effectiveness Depends on Model Capability

GLM-5.1's group D (2.7/7) underperforms groups B and C (both 3.7/7), while GPT-5.4's group C (code-batch, 5.0/7) significantly outperforms group B (step-only, 2.0/7). This cross-model comparison reveals an important insight: the marginal return of code understanding is not fixed but grows with the model's code comprehension ability. The negative interaction effects observed on both models partly stem from the increased cognitive load of simultaneously enabling code analysis and step-wise execution—the context switching between code reading and runtime exploration consumes extra iterations. When the model has stronger code understanding (GPT-5.4) or more efficient tools (LSP semantic navigation replacing text search), the return on investment in the code understanding phase improves significantly.

The practical implication is that as LLM code comprehension capabilities continue to improve, the "code + runtime" combination will gradually become the optimal strategy, and the relative advantage of "runtime-only" mode will diminish. In our GPT-5.4 experiments, we also integrated LSP (gopls) for semantic code analysis, enabling the agent to obtain precise cross-module reference relationships through `lsp_references`, `lsp_definition`, and `lsp_symbols`—a single `lsp_references("Publish")` call returns all 11 event publication points, replacing multiple `search_code` + `read_file` combinations. In larger real-world projects, LSP's advantages will be further amplified.

6.4 Complementarity of Static Analysis and Runtime Testing

The code-only baseline stably discovers approximately 4/7 defects on both models and is the only path to discovering B7 (ID collision)—indicating that static analysis excels at generating "suspicions." However, static analysis cannot confirm or rule out false positives through runtime verification, while runtime testing provides confirmatory evidence. The two are complementary, not substitutes. A promising improvement direction is to use static analysis as a priority ranker for runtime testing—the agent first marks suspicious areas through code analysis, then concentrates limited runtime budget on high-priority areas.

6.5 Implications for Engineering Practice

The method proposed in this paper points toward a new engineering division of labor: developers provide the system, rules, and constraint boundaries; the agent is responsible for continuously understanding the system, exploring test paths, registering missing verification interfaces, executing tests, and producing reports; humans intervene only in rule definition and high-risk conclusion review. From this perspective, integration testing should not merely be about "executing old tests faster" but should gradually shift toward "letting the agent bear the majority of system-level verification labor."

7 THREATS TO VALIDITY

7.1 Internal Validity

Run count and statistical power: Both models' L2 ablation, L0, and L1 experiments completed 3 runs per group (42 total runs). Three repetitions are sufficient to observe stability trends (e.g., all L0/L1 runs across both models detected B1 and B4), but insufficient for rigorous statistical tests such as the Wilcoxon rank-sum test. Future work should scale to 10+ runs to improve statistical power.

Representativeness of seeded defects: The 7 defects were designed and seeded by the authors and may not fully represent the defect distribution in real systems. Defect design referenced common event-driven system fault patterns, but future work should consider mutation testing methodologies to increase objectivity.

Iteration budget effects: GLM-5.1's maximum iteration count was set to 110 and GPT-5.4's to 80; actual iteration consumption varied across groups (25–108), potentially affecting fair comparison of each group's capability upper bound.

7.2 External Validity

System scale: Experiments were conducted on a single prototype system with 6 modules; whether conclusions generalize to larger, more complex systems requires further validation.

Domain generalizability: Although event-driven architectures are widespread in microservices, IoT, and gaming, our experimental system is a game server; applicability to other domains requires additional validation.

Model dependence: Experiments used two models (GLM-5.1 and GPT-5.4), with significant model capability effects already observed (e.g., code understanding main effects differ in direction across models). Two models are insufficient to establish universal conclusions; future work should extend to more models.

7.3 Construct Validity

Defect judgment criteria: Some agent-reported findings require human judgment on whether they constitute true defects. Consistency of judgment criteria may affect the accuracy of precision rates.

8 CONCLUSION AND FUTURE WORK

8.1 Conclusion

This paper presents DSMB-Agent, an agent-driven integration testing method based on deterministic state machine breakpoints. Through experimental validation on two LLMs (GLM-5.1, GPT-5.4), we draw the following conclusions:

(1) Step-wise execution and code understanding each make independent contributions—on GLM-5.1, step-only and code-batch groups both reach 3.7/7 (vs. batch-only 1.3/7); on GPT-5.4, code-batch performs best (5.0/7).

(2) The effectiveness of code understanding depends heavily on model capability—on GLM-5.1, step-wise execution shows a more pronounced marginal benefit (B group 3.7 vs A group 1.3), while on GPT-5.4, code understanding lifts batch mode from 1.7 to 5.0.
(3) Both models exhibit negative interaction effects—simultaneously enabling code analysis and step-wise execution reduces effectiveness, indicating a cognitive load ceiling in the current agent architecture.
(4) B1 is a shared blind spot across all L2 groups on both models—command registration capability (L0/L1) is key to overcoming interface blind spots.
(5) L0 (fully autonomous) mode on both models can build a complete test command set from scratch and discover 3–5 defects; L1 mode on GPT-5.4 averages 5.3/7 (single-run max 6/7), validating cross-model feasibility.
(6) Static analysis and runtime testing are complementary—code-only stably discovers approximately 4/7 defects on both models; L1+LSP on GPT-5.4 achieves comparable detection rates with runtime verification capability.

8.2 Future Work

- **Scale up repeated experiments**—current 3 runs per group can observe trends but should be expanded to 10+ for rigorous statistical testing.
- **Deep LSP integration**—our LSP (gopls) integration is prototypical; future work should systematically evaluate the quantitative impact of LSP semantic navigation on iteration efficiency and defect detection rate.
- **Larger-scale systems**—validate scalability on systems with more modules and more complex interactions.
- **Quantitative comparison with existing methods**—conduct controlled experiments comparing against MBT, random testing, and other baselines.

REFERENCES

[1] Wasif Afzal, Richard Torkar, and Robert Feldt. 2009. A Systematic Review of Search-Based Testing for Non-Functional System Properties. *Information and Software Technology* 51, 6 (2009), 957–976.
[2] Andrea Arcuri and Lionel Briand. 2011. Adaptive Random Testing: An Illusion of Effectiveness?. In *Proc. ISSTA*. 265–275.
[3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374* (2021).
[4] Zhuokui Chen, Lingwei Huang, Yichen Fan, et al. 2023. ChatUniTest: A Framework for LLM-Based Test Generation. *arXiv preprint arXiv:2305.04764* (2023).
[5] Yinlin Deng, Chunqiu Steven Xia, Haoran Peng, Chenyuan Yang, and Lingming Zhang. 2023. Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models. In *Proc. ISSTA*. 1165–1176.
[6] Michael E. Fagan. 1976. Design and Code Inspections to Reduce Errors in Program Development. *IBM Systems Journal* 15, 3 (1976), 182–211.
[7] Google. 2023. UI/Application Exerciser Monkey. Android Developers Documentation. https://developer.android.com/studio/test/other-testing-tools/monkey.
[8] Wolfgang Grieskamp. 2006. Multi-Paradigmatic Model-Based Testing. In *Proc. FATES/RV*. 1–19.
[9] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *Proc. ICSE*. 919–931.
[10] LLVM Project. 2023. libFuzzer – A Library for Coverage-Guided Fuzz Testing. https://llvm.org/docs/LibFuzzer.html.
[11] Ke Mao, Mark Harman, and Yue Jia. 2016. Sapienz: Multi-objective Automated Testing for Android Applications. In *Proc. ISSTA*. 94–105.
[12] Sida Peng, Eirini Kalliamvakou, Peter Cihon, and Mert Demirer. 2023. The Impact of AI on Developer Productivity: Evidence from GitHub Copilot. *arXiv preprint arXiv:2302.06590* (2023).
[13] Ting Su, Guozhu Meng, Yuting Chen, Ke Wu, Weiming Yang, Yao Yao, Geguang Pu, Yang Liu, and Zhendong Su. 2017. Guided, Stochastic Model-Based GUI Testing of Android Apps. In *Proc. ESEC/FSE*. 245–256.
[14] Mark Utting, Alexander Pretschner, and Bruno Legeard. 2012. A Taxonomy of Model-Based Testing Approaches. *Software Testing, Verification and Reliability* 22, 5 (2012), 297–312.
[15] Michal Zalewski. 2014. American Fuzzy Lop (AFL). https://lcamtuf.coredump.cx/afl/.
[16] Xiang Zhou, Xin Peng, Tao Xie, Jun Sun, Chao Ji, Wenhai Li, and Dan Ding. 2021. Fault Analysis and Debugging of Microservice Systems: Industrial Survey, Benchmark System, and Empirical Study. *IEEE Trans. Software Eng.* 47, 2 (2021), 243–260.

A EXPERIMENTAL PROTOTYPE

The prototype source code is available in the `ai-integration-test-demo/` directory, containing the complete server implementation, agent framework, and experiment scripts.

B EXPERIMENT RESULTS DATA

- GLM-5.1 full experiment data (7 groups × 3 runs): `results/formal/glm-5.1/`
- GPT-5.4 full experiment data (7 groups × 3 runs): `results/formal/gpt-5.4/`